HW4 — ezbeda2023 — DRAFT PREVIEW

Reflowed text preview, not a Word PDF export. Screenshots, screencast and live Gemini verification are pending. Export the completed Word document to PDF before Canvas submission.

Security (System) Engineering with Generative AI

Homework Assignment no. 4 — DRAFT: evidence pending

Student name: ezbeda2023

Algorithm:

Edit above your student name and homework number.

Edit below the missing information between < and >.

During development, for every significant task given to the agent you must:

Take a screenshot of your VSCode window clearly showing the prompt you gave to the agent and the response it gave you back. Your FAU ID must be visible on the screenshot.

Take a screenshot of your VSCode window clearly showing at least the top few lines of code that were generated or modified by the agent. Your FAU ID must be visible on the screenshot.

Insert the screenshots into the homework Word document, along with the task prompt given to the agent pasted as regular text.

When finished, convert the Word document to PDF and upload on the homework Canvas page.

Screencast Recording URL: PENDING — record and insert the real YouTube URL.

Github Repository URL: https://github.com/ezbeda2023/gensec-homework (private)

Application name: MCP Security Triage

Application Description: A Gemini agent coordinates an original Python security-triage MCP server and the official published Fetch MCP server. It retrieves Requests release notes, inspects a synthetic asset inventory, ranks potentially affected assets using an explicit classroom heuristic, and assigns proposed upgrades to owners. Both servers use stdio MCP with discovered, namespaced tools. Dependencies are isolated with uv and locked; the Gemini key is provided through an environment variable. A real no-key scripted demo verifies MCP plumbing. Live model verification and genuine development screenshots remain pending. No production systems are scanned or changed.

Development tasks and evidence status

PENDING: insert genuine VS Code screenshots; no images have been fabricated.

Task 1 — Original assignment request

Original task prompt (formatting normalized):

here are my assignment instructions help build this out entierly and create the repo and make sure to push as in the instructions: if you need any more info let me know and I will provide it.

Within your course repository, create a directory for the homework, then commit and push to your remote repository.

cd <path_to_repo> mkdir hw4 touch hw4/agent.py touch hw4/mcp-server.py touch hw4/screencast_url.txt git add hw4 git commit -m "initial commit" git push

In this homework, based on the lab exercises, you will create your own MCP-based agent application using a combination of an MCP server that you write and an MCP server you integrate from published sources.

The MCP server that you write should:

- Implement a set of custom tools for performing functions or returning data or prompts

The MCP server that you integrate should:

- Select MCP servers from the original repository (https://github.com/modelcontextprotocol/servers) or from the more recent MCP Market page (https://mcpmarket.com/server).

CAUTION: because malicious MCP servers are common, investigate the authenticity of the server before installing or running it.

The MCP agent application should

- Configure each MCP server and be able to coordinate the use of each of the tools in them to solve a multi-step problem

Your development should be organized and incremental, with frequent commits into your git repository. Code should also be properly documented via Python docstrings (https://peps.python.org/pep-0257/). Use the uv virtual environment manager to isolate your project dependencies. Use a virtual Python environment as demonstrated in the labs Codelabs. In addition, you must also ensure API keys do not show up in your source files, but rather are passed in via environment variables. Ensure your application code is pushed to your repository before class.

Agentic Code Help and Proper Documentation

For this assignment you must use a coding agent from VSCode (e.g. Claude, Codex). Follow these instructions:

- Download the Homework Template file (https://icardei.github.io/gensec-web/homework-template.docx).

- Rename it to hw3-FAUID.docx, FAUID being your FAU ID.

- During developement, for every significant task given to the agent you must:

- take a screenshot of your VSCode window showing the prompt you gave to the agent and the response it gave you back.

- take a screenshot of your VSCode window showing at least the top few lines of code that were generated or modified by the agent

- the screenshots must show visible your FAU ID or name

- insert these screenshots into the homework Word document, along with task prompt given to the agent.

- Add the missing information between < and > in the Word document.

- When finished, convert the Word document to PDF and upload it on the Homework Canvas page.

Response/result: implemented custom list_assets, assess_assets and remediation_plan tools; integrated the published Fetch server; added a multi-server client, configuration, docstrings and a uv lockfile. Integration testing corrected MCP result decoding and Windows Unicode output.

Evidence pending: original VS Code prompt/response screenshot and code screenshots of agent.py, mcp-server.py and triage.py. Each must show ezbeda2023.

Task 2 — Repository creation

Prompt: “Create a new homework repository under my GitHub account”.

Response/result: created private ezbeda2023/gensec-homework, made the requested initial commit, then incremental implementation and verification commits, and pushed them to main.

Evidence pending: genuine prompt/response screenshot and terminal Git history. No application code was generated specifically for this clarification.

Task 3 — Gemini configuration

Prompt: “I have a gemini api key I can input”.

Response/result: adapted the model loop to Google’s documented OpenAI-compatible Gemini API and added start-agent.ps1 for hidden, temporary key input. No OpenAI account is required.

Evidence pending: prompt/response screenshot and code screenshots of run_agent and start-agent.ps1. Never expose the key.

Verification: the live scripted demo fetched the upstream release history and completed all four tool calls across the two real MCP servers. Tests cover scoring, versions, invalid inputs, destination policy and simulated model orchestration. Live Gemini verification requires the student’s key. See README.md for exact commands.

Before submission: insert screenshots, run the Gemini agent, record the screencast, fill its URL in this document and screencast_url.txt, export the finished PDF, verify the pages, upload to Canvas and grant the instructor repository access.

Filename note: hw3-ezbeda2023.docx follows the supplied naming instruction even though this is Homework 4.